

# Diseño y Desarrollo de Software

LABORATORIO N° 06

## Menú - Navegación



DOCENTE:

**Juan José León  
Suiyon**

CURSO:

**Programación en  
Móviles  
4to Ciclo**

# LABORATORIO 06

## OBJETIVO

- Implementar un DropdownMenu contextual sobre un elemento específico de una lista.
- Personalizar un DropdownMenu con íconos y separadores.
- Implementar un NavigationDrawer como navegación principal de la app, envolviendo un Scaffold existente.
- Personalizar un NavigationDrawer: encabezado de usuario, íconos, e ítem activo resaltado.
- Aplicar control de versiones con GitHub en dos fases: desarrollo propio y mejora asistida por IA.

## II. Equipos y materiales

- Android Studio con soporte Jetpack Compose.
- Proyecto de la TECSUP Store (Laboratorios 4 y 5) como base — si no lo conservas, puedes recrear una versión simplificada.
- Cuenta de GitHub y Git configurado.

## III. Punto de partida

Este laboratorio NO empieza de cero: parte de la TECSUP Store que ya construiste (LazyRow de categorías + LazyColumn de secciones con productos, del Laboratorio 4; navegación entre pantallas, del Laboratorio 5). Tu trabajo es agregarle DOS piezas nuevas que le faltaban:

- Un DropdownMenu en cada tarjeta de producto (menú contextual de 3 puntos).
- Un NavigationDrawer como navegación principal, reemplazando o complementando la forma en que el usuario se mueve entre secciones.

## IV. Metodología de trabajo: GitHub en dos fases

### Fase 1 — Desarrollo SIN IA (rama main)

Agrega el DropdownMenu y el NavigationDrawer a tu TECSUP Store existente, sin usar ningún asistente de IA.

1. Si ya tienes el repositorio de la TECSUP Store, continúa sobre él; si no, crea uno nuevo y vincula tu proyecto.
2. Trabaja sobre la rama main.
3. Mínimo 6 commits descriptivos y distribuidos. Sugerencia de hitos: (1) ícono de 3 puntos + estado expanded en la tarjeta de producto, (2) DropdownMenu con opciones básicas funcionando, (3) personalización del DropdownMenu (íconos, divisores), (4) estructura del NavigationDrawer con ModalDrawerSheet, (5) navegación real desde los ítems del drawer, (6) personalización del drawer (encabezado, ítem activo).
4. Al terminar: git push origin main.

### Fase 2 — Mejora con IA (rama mejora-ia)

Crea la rama mejora-ia A PARTIR de main. Con ayuda de un asistente de IA, agrega la siguiente mejora:

- Mejora obligatoria: agrega un badge con contador en el ítem "Favoritos" del drawer (mostrando cuántos productos marcó el usuario como favorito desde el DropdownMenu de cada producto). Esto conecta las dos piezas del laboratorio: una acción en el DropdownMenu debe reflejarse visualmente en el Drawer.
1. git checkout main
  2. git checkout -b mejora-ia
  3. Mínimo 3 commits descriptivos.
  4. Documenta cada prompt usado en un archivo PROMPTS.md.
  5. Al terminar: git push origin mejora-ia.

## V. Caso propuesto

Resultado esperado — DropdownMenu abierto sobre un producto (izquierda) y NavigationDrawer con navegación principal (derecha):

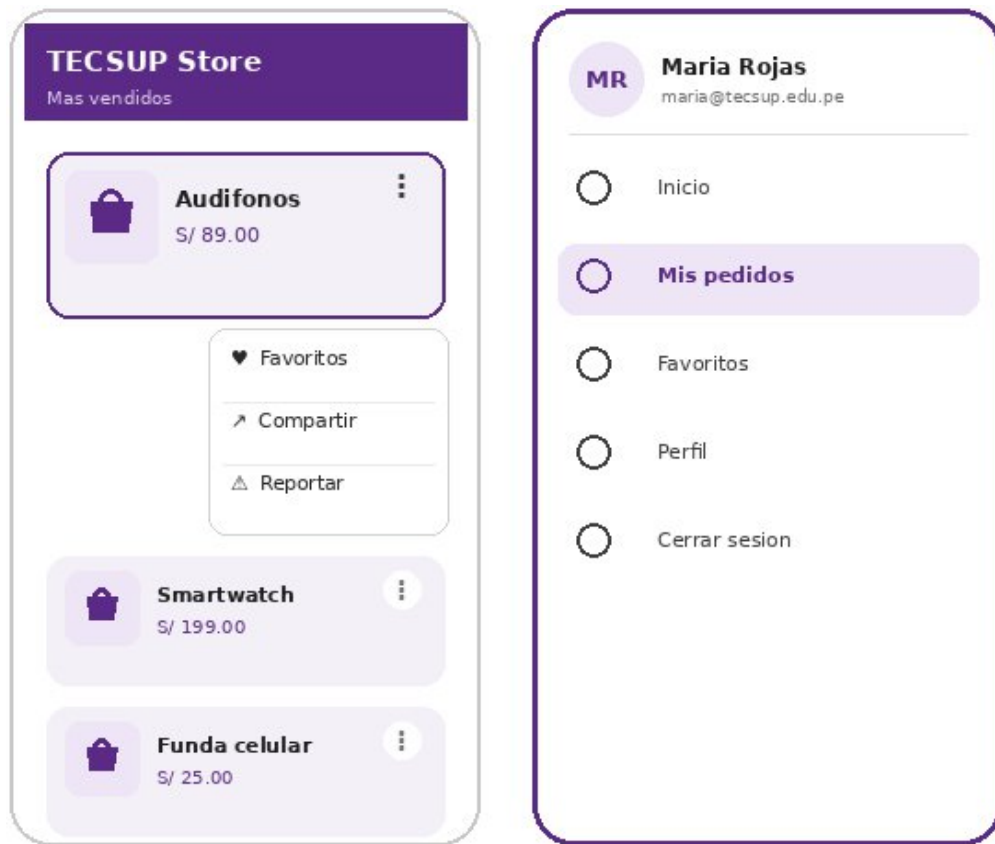


Figura 1. DropdownMenu contextual en un producto y NavigationDrawer de navegación principal.

### Requisitos funcionales

- Cada tarjeta de producto (en cualquier sección de la TECSUP Store) tiene un ícono de 3 puntos (:) a la derecha.
- Al tocarlo, se despliega un DropdownMenu con mínimo 3 opciones: "Favoritos", "Compartir", "Reportar" (o equivalentes de tu criterio).
- Cada opción del DropdownMenu tiene su ícono correspondiente (leadingIcon).
- Un ícono ≡ en la topBar abre un NavigationDrawer con mínimo 4 destinos: Inicio, Mis pedidos, Favoritos, Perfil (+ Cerrar sesión opcional).
- El encabezado del drawer muestra el avatar/iniciales y datos básicos del usuario.
- El destino activo del drawer se resalta visualmente (color de fondo distinto al resto).

### Estructura de archivos sugerida (agregado a lo ya existente)

|                    |                                                     |
|--------------------|-----------------------------------------------------|
| TarjetaProducto.kt | → agrega el ícono de 3 puntos + DropdownMenu        |
| AppDrawer.kt       | → contenido del NavigationDrawer (ModalDrawerSheet) |
| AppNavegacion.kt   | → se envuelve con ModalNavigationDrawer             |

## VI. Preguntas de reflexión

- ¿Por qué el DropdownMenu se declara dentro de un Box junto al ícono que lo activa, y no en cualquier parte de la pantalla?
- ¿Qué diferencia de alcance hay entre las opciones del DropdownMenu (afectan solo a un producto) y las del NavigationDrawer (afectan a toda la app)?

- ¿Cómo tuviste que estructurar tu código para que el contador de favoritos del drawer "se entere" de lo que pasa en el DropdownMenu de cada producto?
- ¿Qué tuviste que corregir del código que te generó la IA para la mejora del badge de favoritos?

## VII. Observaciones y conclusiones

Elaboración individual. Mínimo 2 observaciones y 2 conclusiones.

- Observaciones: notas aclaratorias, dudas u obstáculos durante el desarrollo.
- Conclusiones: opinión personal sobre el trabajo realizado, comparando el proceso de la Fase 1 y la Fase 2.

## Tarea: Clínica SaludPlus — App Paciente

Oct 2, 2026 · @C24

Complementaria al Laboratorio 6, a partir de un código esqueleto.

### I. Objetivos

- Completar una app real de agendamiento de citas médicas a partir de un código esqueleto, llenando cada archivo .kt según sus comentarios TODO.
- Implementar NavigationBar como menú principal de navegación en la pantalla de Inicio (Inicio, Citas, Resultados, Perfil).
- Aplicar LazyRow (especialidades destacadas), LazyColumn (especialidades, médicos y citas) y LazyVerticalGrid (horarios) trabajando con colecciones en memoria.
- Aplicar navegación con paso de parámetros (especialidadId, medicoId, fecha, hora) y popUpTo.
- Diseñar las vistas que faltan en el diseño de referencia respetando su estilo visual.
- Aplicar control de versiones con GitHub en dos fases: desarrollo propio y mejora asistida por IA.

### II. Punto de partida: código esqueleto

Esta tarea NO empieza en blanco. Se entrega el proyecto SaludPlusCitas.zip, que se abre directamente en Android Studio (File > Open). El proyecto ya compila y navega: cada pantalla muestra un contenido temporal (PantallaEnConstruccion) con botones para pasar a la siguiente.

- **Entregados COMPLETOS** (no son el objetivo de aprendizaje): MainActivity.kt, los 4 modelos (Usuario, Especialidad, Medico, Cita), el tema (Color.kt, Theme.kt, Type.kt), Rutas.kt y AppNavigation.kt.
- **ESQUELETO** (debes completar cada TODO): Repositorio.kt y las 15 pantallas.
- **Por crear:** los componentes reutilizables del paquete ui/components (Componentes.kt sugiere cuáles).

Al terminar cada pantalla, borra la llamada a PantallaEnConstruccion. Para ver todo lo pendiente usa View > Tool Windows > TODO.

Reglas: no usar base de datos (ni Room, ni SQLite, ni Firebase), no cambiar nombres ni parámetros de las funciones del Repositorio ni de las pantallas, y no modificar AppNavigation.kt salvo para agregar pantallas nuevas.

### III. Estructura del desarrollo

El proyecto se organiza por capas (datos, navegación, interfaz) y las pantallas por módulo funcional. Cada pantalla vive en su propio archivo.

#### Paquetes

com.saludplus.citas

```
├─ MainActivity.kt          (completo)
├─ data
│   └─ model                (completo)
│       ├── Usuario.kt
│       ├── Especialidad.kt
│       ├── Medico.kt
│       └─ Cita.kt
├─ repository
│   └─ Repositorio.kt      (esqueleto: colecciones + funciones TODO)
├─ navigation              (completo)
│   └─ Rutas.kt
├─ AppNavigation.kt
└─ ui
    ├── theme                (completo)
    ├── components           (por crear)
    └─ screens               (esqueleto)
        ├── auth             SplashScreen, RegistroScreen, LoginScreen, TerminosScreen
        ├── home              HomeScreen
        ├── agendamiento      EspecialidadesScreen, MedicosScreen, FechaHoraScreen,
        │                     ConfirmarCitaScreen, CitaExitosaScreen
        ├── citas             MisCitasScreen, DetalleCitaScreen
        ├── perfil            PerfilScreen
        ├── resultados        ResultadosScreen
        └─ notificaciones     NotificacionesScreen
```

#### Pantallas

El diseño trae 7 pantallas. Los botones del diseño llevan a 4 vistas más que no están dibujadas y son obligatorias; otras 4 quedan como reto extra.

| # | Pantalla       | Archivo                              | Tipo     | Qué se practica                                          |
|---|----------------|--------------------------------------|----------|----------------------------------------------------------|
| 1 | Splash         | auth/SplashScreen.kt                 | Diseño   | Image, Column, botones                                   |
| 2 | Registro       | auth/RegistroScreen.kt               | Diseño   | Estados, OutlinedTextField, validaciones, add a la lista |
| 3 | Inicio         | home/HomeScreen.kt                   | Diseño   | Scaffold, NavigationBar, LazyRow                         |
| 4 | Especialidades | agendamiento/EspecialidadesScreen.kt | Diseño   | LazyColumn, búsqueda con filter                          |
| 5 | Médicos        | agendamiento/MedicosScreen.kt        | Diseño   | Parámetro especialidadId, filter + sortedByDescending    |
| 6 | Fecha y hora   | agendamiento/FechaHoraScreen.kt      | Diseño   | LazyVerticalGrid, selección, horarios disponibles        |
| 7 | Confirmar cita | agendamiento/ConfirmarCitaScreen.kt  | Diseño   | 3 parámetros, crear y guardar la cita                    |
| 8 | Iniciar sesión | auth/LoginScreen.kt                  | Faltante | Búsqueda con find,                                       |



Flujo de navegación · 13 pantallas con parámetros

Las flechas llevan el parámetro que recibe la pantalla siguiente. Al confirmar, popUpTo borra el flujo de agendamiento del historial. También están conectados: Cita agendada → Mis citas, Perfil → Splash (cerrar sesión), la campana de Inicio → Notificaciones y el enlace del Registro → Términos.

#### Orden de trabajo sugerido

1. Implementar las funciones de Repositorio.kt.
2. Crear los componentes reutilizables en ui/components.
3. Completar las pantallas en el orden del flujo: 1 → 2 → 8 → 3 → 4 → 5 → 6 → 7 → 9 → 10 → 11.
4. Retos extra (12 a 15).

### IV. Metodología de trabajo: GitHub en dos fases

#### Fase 1 — Desarrollo SIN IA (rama main)

Completa TODOS los TODO obligatorios del esqueleto sin usar ningún asistente de IA.

1. Crea el repositorio en GitHub y sube el código esqueleto como primer commit.
2. Trabaja sobre la rama main.
3. Mínimo 8 commits descriptivos y distribuidos. Hitos sugeridos:
  - a. Repositorio: usuarios (registrarUsuario, iniciarSesion, cerrarSesion).
  - b. SplashScreen + RegistroScreen + LoginScreen con validaciones.
  - c. Repositorio: especialidades y médicos (búsquedas y filtros).
  - d. HomeScreen con tarjetas, saludo y LazyRow de especialidades destacadas.
  - e. NavigationBar (bottomBar) con 4 destinos funcionando.
  - f. EspecialidadesScreen con búsqueda + MedicosScreen con parámetro especialidadId.
  - g. FechaHoraScreen con LazyVerticalGrid y horariosDisponibles.
  - h. ConfirmarCitaScreen + CitaExitosaScreen + popUpTo.
  - i. MisCitasScreen + PerfilScreen.
4. Al terminar: git push origin main.

#### Fase 2 — Mejora con IA (rama mejora-ia)

Crea la rama mejora-ia A PARTIR de main. Con ayuda de un asistente de IA, agrega la siguiente mejora.

**Mejora obligatoria: calendario dinámico en la Pantalla 6 (Fecha y hora).** En la Fase 1 los días se muestran como una lista fija. En esta fase deben generarse con java.time.LocalDate:

- Mostrar los próximos 5 días hábiles a partir de hoy (sin sábados, domingos ni días pasados).
- Las flechas < y > avanzan o retroceden una semana; no se puede retroceder antes de la semana actual.
- El nombre del mes y año ("Octubre 2026") cambia según la semana mostrada.
- Al cambiar de día, los horarios disponibles se recalculan solos y la hora seleccionada se reinicia.
- La Pantalla 7 muestra la fecha en texto en español ("Martes 16 de setiembre 2026").

Ambas partes deben seguir funcionando juntas: el calendario dinámico no debe romper el bloqueo de horarios ya reservados.

1. git checkout main
2. git checkout -b mejora-ia
3. Mínimo 3 commits descriptivos.
4. Documenta cada prompt usado en un archivo PROMPTS.md (prompt, respuesta resumida y qué tuviste que corregir).
5. Al terminar: git push origin mejora-ia.

### V. Rúbrica de evaluación (20 puntos)

| Criterio                          | Descripción                                                                                                                           | Pts |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-----|
| Asistencia                        | Presencial y puntual a la sesión de laboratorio.                                                                                      | 1   |
| Puntualidad de entrega            | Entrega el informe dentro del plazo indicado por el docente.                                                                          | 1   |
| Estructura de archivos completada | Todos los TODO obligatorios resueltos; cada pantalla en su archivo y paquete; sin PantallaEnConstruccion en las pantallas terminadas. | 2   |
| Repositorio con colecciones       | Todas las funciones de Repositorio.kt implementadas con operaciones de colecciones, sin base de datos.                                | 2   |
| Registro, login y sesión          | Registro con validaciones, login contra la lista de usuarios, saludo con el nombre y cerrar sesión en                                 | 1   |

| Criterio                              | Descripción                                                                                                                                                                   | Pts       |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
|                                       | Perfil.                                                                                                                                                                       |           |
| Flujo de agendamiento completo        | Inicio → Especialidades → Médicos → Fecha y hora → Confirmar → Cita agendada, usando los parámetros recibidos (especialidadId, medicoId, fecha, hora) y popUpTo al confirmar. | 3         |
| NavigationBar (menú, Semana 6)        | bottomBar funcional con 4 destinos en Inicio: Inicio, Citas, Resultados, Perfil.                                                                                              | 2         |
| LazyRow y LazyColumn                  | LazyRow de especialidades destacadas; LazyColumn de especialidades con búsqueda en tiempo real, de médicos y de mis citas (con mensaje de lista vacía).                       | 2         |
| Horarios reactivos                    | LazyVerticalGrid de horarios; un horario reservado deja de aparecer para ese médico y fecha; Continuar solo se habilita con día y hora elegidos.                              | 2         |
| Fase 1 — Commits en main              | Mínimo 8 commits descriptivos y distribuidos durante el desarrollo SIN IA.                                                                                                    | 2         |
| Fase 2 — Calendario dinámico (CON IA) | Días hábiles reales con LocalDate, flechas por semana, mes dinámico y horarios recalculados al cambiar de día.                                                                | 1         |
| Fase 2 — Commits y PROMPTS.md         | Mínimo 3 commits en mejora-ia; PROMPTS.md completo y coherente.                                                                                                               | 1         |
| <b>TOTAL</b>                          |                                                                                                                                                                               | <b>20</b> |

Los retos extra (pantallas 12 a 15) no suman puntos de la rúbrica; el docente puede usarlos para compensar puntos perdidos.

## VI. Diseño de referencia y alcance

Las 7 pantallas del diseño con el flujo completo de agendamiento:



Figura 1. App Paciente — Clínica SaludPlus. Las vistas faltantes (Login, Cita agendada, Mis citas, Perfil) se diseñan con el mismo estilo: colores, tarjetas, botones azules y barra superior con flecha.

**Nota importante sobre el alcance.** Esta tarea construye SOLO la App Paciente. No hay conexión con un sistema de la clínica ni con una base de datos: usuarios, especialidades, médicos y citas viven en colecciones dentro del objeto Repositorio y se pierden al cerrar la app. Esto es intencional: con lo visto hasta la Semana 6 (sin MVVM formal ni Room) no corresponde todavía una persistencia real.

## VII. Preguntas de reflexión

- ¿Por qué los modelos, Rutas.kt y AppNavigation.kt se entregaron completos y las pantallas no? ¿Qué tienen en común los archivos que sí se dejaron como esqueleto?
- ¿Por qué el Repositorio es un object y no una clase normal? ¿Qué pasaría con las citas si cada pantalla creara su propia lista?
- ¿Cómo lograste que la búsqueda de especialidades y los horarios disponibles se actualicen solos, sin que tú "actualices" nada a mano?
- ¿Qué diferencia notaste entre navigate() normal (Especialidades → Médicos) y el que usa popUpTo (Confirmar cita → Cita agendada)? ¿Qué pasa al presionar Atrás en cada caso?
- ¿Qué tuviste que corregir del código que te generó la IA para el calendario dinámico?
- Compara el NavigationDrawer del Laboratorio 6 con el NavigationBar de esta tarea: ¿en qué caso usarías cada uno en un proyecto propio?

## VIII. Observaciones y conclusiones

Elaboración individual. Mínimo 2 observaciones y 2 conclusiones.

- **Observaciones:** notas aclaratorias, dudas u obstáculos durante el desarrollo, incluido el diseño de las vistas faltantes.
- **Conclusiones:** opinión personal sobre trabajar a partir de un esqueleto en vez de empezar desde cero, comparando el proceso de la Fase 1 y la Fase 2.